

Reverse Engineering the Monolithic Engine of Dragon Warrior VII: Memory Partitioning, Event Flag Matrices, and Asset Compression Protocols

Memory Partitioning and Heap Allocation Mapping

The execution of a massive, multi-disc role-playing game on the fifth-generation PlayStation platform forced developer Heart Beat to bypass standard operating system APIs in favor of a custom, highly optimized system architecture. With only 2\text{ MB} (2048\text{ KB}) of system RAM available, the main executable SLUS_012.06 (and its Japanese counterpart SLPM_865.00) maps out a strict partitioning layout. This layout separates the static, resident "base engine" from volatile "expansion content" and high-bitrate data streamed dynamically from the optical disc.

The lower bounds of system memory host the resident base engine, which includes core MIPS R3000 CPU instructions, geometry pipeline interfaces, peripheral input maps, and the main gameplay execution loop. Immediately above these system bounds lies the game state tracking matrix, followed by dedicated sound processing software. Because the standard Sony PsyQ sound library was too generic and resource-intensive, Heart Beat implemented a custom sound driver, designated SEQq, which utilizes the signature "qQES". This custom driver bypasses the standard SEQ/VAB protocols to optimize the utilization of both system RAM and Sound RAM (SPU WRAM).

PlayStation System Memory Map Layout

The partition boundaries mapped within the PlayStation's 2\text{ MB} physical memory space are structured as follows:

Physical Address Range	Segment Classification	Core Engine Function / Subsystem	Citations
0x80000000 - 0x8000F7FF	Kernel & System Core	PlayStation operating system kernel vectors, PsyQ base configurations	
0x8000F800 - 0x800252FF	Global Flag Space	Active progress registers, character attributes, event triggers, save state buffer	
0x80025300 - 0x800258E8	Base Sound Engine	Structural interpretation tables for the custom SEQq music driver	

Physical Address Range	Segment Classification	Core Engine Function / Subsystem	Citations
0x8002FA10 - 0x80036FCC	Base Sound Engine	Execution thread code (interrupt-driven sound processor)	
0x80037000 - 0x8007603F	Core Executable	Main game loop, rendering pipelines, dialogue engines, camera systems	
0x80076040 - 0x8008247F	Disk File System	Core optical disc filesystem routines, logical block addressing (open_file)	
0x80082480 - 0x800849D4	Disk File System	Sound streaming program code, low-level mopen_file file pointer tracker	
0x80090E54 - 0x80094D14	Base Sound Engine	PsyQ SPU interface library calls and low-level voice assignment code	
0x800B9D10 - 0x800BB6E7	SPU Work RAM	SPU library parameter storage, hardware voice registers	
0x800D25C0 - 0x800D80EF	Volatile Sequence Heap	BGM Sequence streaming buffer (maximum size allocated: 23,344\text{bytes})	
0x800D80F0 - 0x800DA0F7	Volatile Sequence Heap	Instrument attribute maps, panning indices, envelope registers	
0x800F7798 - 0x800F7799	Diagnostic Segment	Master debug mode registration address (16\text{-bit} switch)	
0x80138000 - 0x8017FFFF	Dynamic Heap Segment	Executing sequence (SEQq) play buffer, active script parameters	
0x80180000 - 0x801FFFFFF	Dynamic Heap Segment	SPU ADPCM soundbank (VAB) sample buffer (Waveforms)	

The custom SEQq sound format starts with a 60[span_16](start_span)[span_16](end_span)\text{-byte} header, which maps the layout of incoming audio waveforms and sequence pointers directly into the dynamic heaps. Unlike the standard PsyQ sound format, sequence data parsed by the SEQq driver terminates abruptly

with the byte sequence FF 2F 00 without any trailing delta-time parameters, saving memory overhead in the dynamic heap.

Custom SEQq Sound Format Header Layout

The 60\text{-byte} sound format header is structured as follows:

Byte Offset	Size (Bytes)	Field Name	Technical Description	Citations
0x00	4	SEQ Size	Size of the music sequence payload in bytes (evaluates to 0 if absent)	
0x04	2	SEQ ID	Micro-sequence identifier (msq_id)	
0x06	1	Bank Count	Total number of ADPCM wave banks allocated (typically 4)	
0x07	1	Load Position start_spanend_span	Memory position indicator (0 = Normal BGM, loads to static address)	
0x08	4	Reserved	Reserved space (unutilized by the driver)	
0x0C	4	Bank 1 VBSi start_spanend_span	Compressed ADPCM waveform size (0 if preloaded bank is active)	
0x10	4	Bank 1 Regional Mapping Tables start_spanend_span	Size of the regional mapping tables for Bank 1	
0x14	2	Bank 1 ID	Instrument index descriptor for voice mapping	
0x16	2	Bank 1 Attributes start_spanend_span	Voice panning, pitch modulation, and resonance	

Byte Offset	Size (Bytes)	Field Name	Technical Description	Citations
		nd_span)[span_57](start_span)[span_57](end_span)but e	parameters	
0x18 - 0x23	12	Bank [span_44](start_span)[span_44](end_span)[span_58](start_span)[span_58](end_span)2 Header	Configuration block for Bank 2 (identical structure to Bank 1)	
0x24 - 0x2F	12	Bank [span_45](start_span)[span_45](end_span)[span_59](start_span)[span_59](end_span)3 Header	Configuration block for Bank 3 (identical structure to Bank 1)	
0x30 - 0x3B	12	Bank [span_46](start_span)[span_46](end_span)[span_60](start_span)[span_60](end_span)4 Header	Configuration block for Bank 4 (identical structure to Bank 1)	

Dynamic Map Block Loading and Streaming Architecture

Movement between the world map and individual town zones requires real-time loading routines to prevent memory overflows. The central function that coordinates this operation is located at address 0x80076040 (open_file). This routine acts as an interface to the PlayStation's CD-ROM controller. When the player transitions between map sectors, open_[span_18](start_span)[span_18](end_span)file invokes a secondary, non-blocking sub-loader at address 0x80082598 (mopen_file).

The mopen_file routine operates as follows:

1. It accepts a single parameter passed through CPU register \$a0, which points to a file descriptor struct. This struct contains the target Logical Block Number (LBN), sector length, and the destination address in the dynamic heap.
2. Rather than attempting to copy large data blobs in a single processing frame—which would stall the main game loop—the function reads the file from the disc incrementally, sector-by-sector, in 2048\text{-byte} chunks.
3. For map instances, the sub-loader detects the resource type as 0x13, prompting the engine to partition the dynamic loading process across multiple frames.
4. The system leverages DMA Channel 4 (the CD-ROM direct memory access pathway) to stream sectors directly into the dynamic map geometry heap starting at 0x80138000, bypassing main CPU cycles.

This streaming approach has a noticeable impact on emulators. When the game menu is opened, the engine takes a snapshot of the current frame buffer from VRAM and copies it to a dedicated background buffer in System RAM. Because of strict memory constraints, this copy is captured at the game's native internal rendering resolution (typically 256×224 or 320×240 pixels). On original hardware, this transfer is seamless. However, on modern emulators running at upscaled resolutions, this process creates a noticeable rendering discrepancy: the high-resolution menu elements are overlaid on top of a low-resolution, blocky background snapshot, resulting in a distinct "blurry" menu effect.

Event Trigger and Flag Matrix Architecture

To maintain a persistent, non-linear narrative across dozens of islands existing in distinct past and present time periods, the Heart Beat engine utilizes a compact, highly dense game state tracking system. This tracking is managed by an event flag matrix residing in the system's static memory bounds.

Tracing the Progression Flags in RAM

Live memory tracing on a PlayStation emulator locates the core game progress flags within the RAM range 0x8000F800 through 0x80010000. This space acts as a bit-packed matrix that records every major progression state :

- Active system flags are modified by the engine's state-writing routines. For example, the "Save Anywhere" debug codes write directly to the flag ranges 0x8000F83C through 0x8000F846.
- Equipment rules and character restrictions are managed by related addresses in this block; for instance, the "Equip Any Armor" bypass writes directly to static address 0x80069BCA to override standard inventory validation checks.

To trace these state changes in a debugging environment (such as No\$psx), developers can set a write breakpoint directly on the target flag address using the syntax [0x8000F83C]!. When an event occurs—such as solving a puzzle in the Estard Ruins or recruiting an immigrant—the engine executes a store byte instruction, typically:

```
sb v1, $0(a1)
```

This instruction writes the updated state byte to the flag pointer held in register \$a1. To test flag bypasses or manipulate state transitions, the conditional branch instruction that evaluates these flags:

```
beq v0, a0, $8001129C
```

can be patched in memory with a No-Operation (nop) instruction to prevent the jump, or replaced with an unconditional jump (j \$8001129C) to force the game state to evaluate as true regardless of the actual flag values.

Dynamic Town Streaming and Flag-Driven Model Selection

The engine uses this progress flag matrix to dynamically stream 3D models and construct towns in real-time. The most prominent example is the Immigrant Town, which dynamically alters its physical layout, loaded buildings, and active NPCs based on the current population flag. When

the player enters the town's sector, the engine's loading routine queries the flag matrix at 0x8000F800 to determine which assets to stream into the dynamic heap.

Immigrant Town Flag-Driven Progression States

The structural shifts and model assets loaded for each town phase are outlined below:

Active Population Flag Value	Structural Phase	Loaded 3D Model Assets and NPC Configurations	Citations
1 - 4 Immigrants	Phase 1	Campsite terrain mesh, base tents, Sim NPC model	
5 - [span_100](start_span)[span_100](end_span) 9 Immigrants	Phase 2	Small wooden shack model, dynamic naming system initialized	
10 - [span_101](start_span)[span_101](end_span) 14 Immigrants	Phase 3	Multi-room building models, raw construction frame meshes	
15 - [span_102](start_span)[span_102](end_span) 19 Immigrants	Phase 4	Basic storefront textures, initial merchant NPC assets	
20 - [span_103](start_span)[span_103](end_span) 24 Immigrants	Phase 5	Expanded Inn model, town management interface subroutine active	
25 - [span_104](start_span)[span_104](end_span) 29 Immigrants	Phase 6	Stone church model, secondary shop models, religious NPC assets	
30 - [span_105](start_span)[span_105](end_span) 34 Immigrants	Phase 7	Bank vault structures, high-tier armor and weapon merchant models	
35 - [span_106](start_span)[span_106](end_span) 40 Immigrants	Phase 8	Final town layout (varies dynamically based on immigrant classes recruited)	

This progression system operates through a specialized flag-to-asset pointer translation function :

1. Upon boundary collision detection (the transition from world map to town instance), the engine suspends the world map renderer.
2. The asset selector function queries the progress flag range 0x8000F800 to retrieve the current state values.
3. The returned state byte acts as an offset modifier for the asset lookup tables stored in

SLUS_012.06. For example, if the population flag reads 22, the selector matches the index to the Phase 5 asset group.

4. This group contains sector addresses for the specific Inn models, textures, and NPC scripts located in the HBD1PS1D.Q71 archive.
5. These addresses are passed directly to `mopen_file` (0x80082598), which streams the selected assets into the dynamic heap at 0x80138000. This process ensures that assets from other phases are never loaded, keeping memory usage well within the PlayStation's 2\text{ MB} limit.

Archive Layout and Compression Protocols

The entire asset library of *Dragon Warrior VII* is contained within the massive, monolithic resource archive HBD1PS1D.Q71. This file spans several hundred megabytes and is divided into strict 2048\text{-byte} blocks to align perfectly with the physical sectors of the PlayStation's CD-ROM drive.

Archive Structural Layout

The HBD1PS1D file format uses a structured hierarchy designed for high-performance optical seeking, divided into distinct block classifications :

Header Block

The first 2048\text{-byte} sector functions as the primary volume descriptor. The ASCII string `hdb1ps1d.q71` is located at byte offset 0x400 (1024\text{ bytes}), serving as the game's archive identification signature.

Primary Data Blocks (* 00 00 00 Blocks)

These blocks act as index directories for grouped assets. Each directory block begins with a 16\text{-byte} header :

- **Offset 0x00 (4 bytes):** Sub-block Count. Specifies the number of asset sub-blocks contained within this folder (capped at approximately 18 sub-blocks).
- **Offset 0x04 (4 bytes):** Sector Count. The total number of physical 2048\text{-byte} sectors occupied by this entire block on the disc.
- **Offset 0x08 (4 bytes):** Total Data Length. The unpadded byte length of the raw, concatenated sub-block payload.
- **Offset 0x0C (4 bytes):** Reserved. Structured as a null dword (0x00000000).

Asset Sub-blocks

Each primary data block contains contiguous asset payloads. The metadata for these assets is defined in a 16\text{-byte} sub-header located at offset $0x10 + (i * 16)$ within the primary block, where i represents the sub-block index :

- **Offset 0x00 (4 bytes):** Compressed Data Size. Denotes the size of the asset payload as stored on the physical disc.
- **Offset 0x04 (4 bytes):** Uncompressed Data Size. Denotes the absolute allocation size

required in system RAM when decompressed.

- **Offset 0x08 (4 bytes):** Unknown system flag.
- **Offset 0x0C (2 bytes):** Compression Flag. If this value is 1280 (0x0500 in hexadecimal), the sub-block's payload is compressed using Heart Beat's custom LZSS0 algorithm. Any other value indicates uncompressed data.
- **Offset 0x0E (2 bytes):** Resource Type Identifier. Identifies the asset classification (such as font graphics, menu assets, 3D character meshes, or map tilemaps).

Control Blocks (0x60010108 Blocks)

Interspersed throughout the archive are 2048\text{-byte} control blocks designed to interface with the CD-ROM drive controller. These blocks are identified by the magic number 0x60010108 in their 32\text{-byte} header :

Header Offset	Size (Bytes)	Field Value / Mapping	Technical Description	Citations
0x00	4	0x60010108	Control block magic signature	
0x04	2	0x0000 - 0x0004	Block index register (increments sequentially per block)	
0x06	2	0x0005	Count constant (always 5)	
0x08	4	Variable Counter	Part counter (increments by 1 each time the block index resets)	
0x0C	4	Integer Value	Internal system tracking integer	
0x10	2	0x8000	SPU interface configuration constant (128)	
0x12	2	0x7800	Audio stream mapping constant (120)	
0x14	4	0xFFFFFFFF38	Hardware register offset (fourth byte locked to 0x38)	
0x18	2	0x0100 / 0x0200 / 0x0300	System execution mode flag	
0x1A	4	0x03000000	Main interface driver constant	
0x1E	2	0x0000	Static null padding	

LZSS0 Decompression Protocol

When the compression flag at offset 0x0C of an asset sub-header reads 1280, the engine

passes the payload to a hardware decompressor routine. The decompressor initializes a sliding dictionary ring buffer of 4096\text{ bytes}. This buffer is initialized entirely with zero (0x00) bytes. The input stream is read as a sequence of control blocks, starting with a 1\text{-byte} control word, C. The decompressor parses this control byte bit-by-bit from the least significant bit (b_0) to the most significant bit (b_7) to process the compressed payload :

\text{Bit } b_i \text{ of control byte } C \implies \begin{cases} 1 & \text{Literal Byte Copy} \\ 0 & \text{Dictionary Reference (16-bit Descriptor } W) \end{cases}

Literal Byte Copy (b_i = 1)

The decompressor reads the next raw byte directly from the input stream, writes it to the destination output buffer in RAM, and appends it to the sliding dictionary history window:

\text{Output} \leftarrow \text{Input}[I] \quad D \leftarrow D + 1, \quad I \leftarrow I + 1

Dictionary Reference (b_i = 0)

The decompressor reads a 2\text{-byte} (16\text{-bit}) descriptor word, W, from the input stream:
 $W \leftarrow (\text{Input}[I] \ll 8) \mid \text{Input}[I+1] \quad I \leftarrow I + 2$

This 16\text{-bit} word is unpacked to extract the history ring-buffer offset, O, and the repeating match length, L:

$O \leftarrow W \gg 4 \quad L \leftarrow (W \& \text{0x0F}) + 3$

The decompressor then copies L bytes from the sliding history buffer starting at offset O directly into the active destination RAM buffer, updating the dictionary ring buffer with the copied bytes.

Input Stream:

```

+---+---+---+---+---+---+
| C |   W   |...| (C = Control Byte, W = 16-bit Descriptor)
+---+---+---+---+---+---+

```

```

|
+--> b_i = 0 (Reference Flag)
|
+--> Decode W:
      Offset (O) = W >> 4
      Length (L) = (W & 0x0F) + 3

```

```

|
| Retrieve L Bytes

```

v

Sliding Ring Buffer (Initialized to 0x00, Size = 4096 Bytes):

```

+-----+
|           [O]---->|
+-----+

```

```

|
+--> Append to Buffer End & Output

```

v

Destination Output RAM Buffer:

```

+-----+---+---+---+
|           | | | |
+-----+---+---+---+

```

Heart Beat's decision to use the LZSS0 algorithm represents a deliberate technical trade-off. Because the PlayStation's CPU lacks a dedicated hardware decompression coprocessor (unlike modern systems or its video-only MDEC chip), on-the-fly decompression of highly packed assets introduces significant processing overhead. LZSS0 requires minimal CPU cycles for decompression, relying on simple bitwise shifts and memory copies. By accepting slightly lower compression ratios compared to more advanced algorithms, the engine is able to decompress graphics, scripts, and map data on-the-fly without causing frame-rate stutter or disrupting the interrupt-driven sound processing threads.

Isometric Dungeon Tile-set Decompression and Assembly

This decompression pipeline is essential for assembling the game's isometric dungeon environments. These environments are built using a collection of 16×16 pixel tiles, with walls often rendered at a height of 32 pixels to establish isometric depth and perspective.

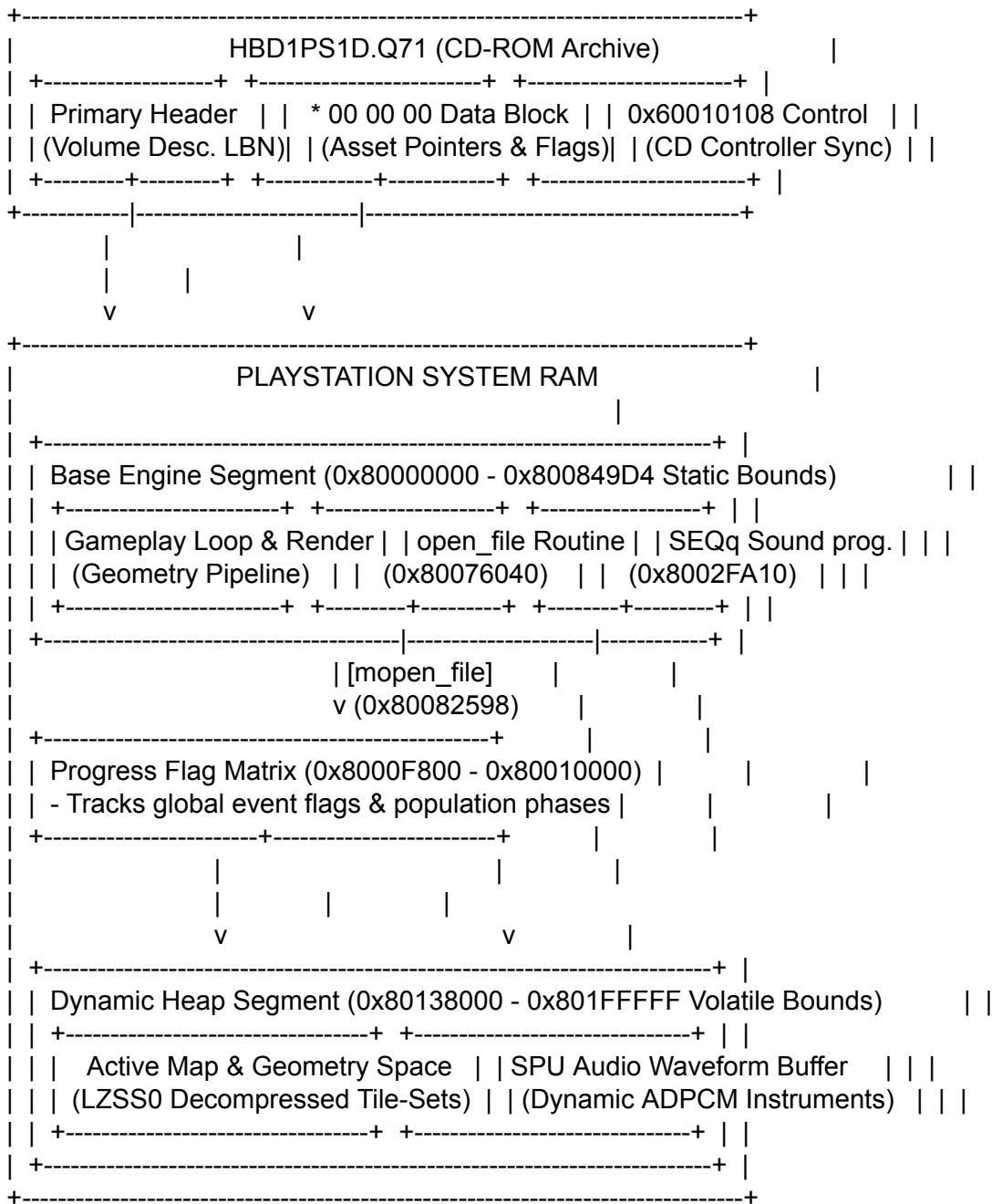
The dungeon assembly sequence proceeds as follows :

1. The engine calls `open_file` to locate the target dungeon directory block inside HBD1PS1D.Q71.
2. Two distinct sub-blocks are loaded: one containing the compressed 16×16 texture tile-sets, and another containing the grid-based isometric layout map.
3. The texture payload is processed by the LZSS0 decompressor, which inflates the compressed data into a raw, linear 4-bit or 8-bit indexed bitmap array representing color indices mapped to local palette tables.
4. The layout sub-block is decompressed to reconstruct a two-dimensional grid of cell pointers. Each cell pointer is a 16-bit descriptor containing the following structural metadata:
 - **Bits 0-9:** Tile Index (points directly to the decompressed 16×16 texture array).
 - **Bit 10:** Horizontal Texture Flip Flag (toggles horizontal mirroring).
 - **Bit 11:** Vertical Texture Flip Flag (toggles vertical mirroring).
 - **Bits 12-15:** Elevation and Height Parameters (defines the isometric layer coordinate, determining whether the tile is rendered at base level or elevated to form wall structures).
5. The rendering engine parses this layout grid sequentially. To construct the isometric perspective, any tile containing an active elevation parameter has its vertical draw coordinate offset by a fixed rate of 4 to 6 pixels. This adjustment ensures that characters and objects align correctly with the floor tiles rather than floating on edge boundaries, creating an accurate sense of scale and depth within the engine's isometric environments.

Technical Synthesis of Core Systems

The custom engine developed by Heart Beat for *Dragon Warrior VII* shows how software-level optimizations can overcome the physical limitations of legacy hardware. By implementing a highly structured and coordinated resource pipeline, the developers successfully managed a massive game footprint within the PlayStation's 2 MB memory limit.

Overview of Resource Management and Processing Pipeline



This structural architecture functions as an integrated pipeline :

- File operations are organized into sector-aligned physical blocks to maximize CD-ROM drive efficiency and reduce head seek times.
- High-bitrate data streaming is controlled by the asynchronous mopen_file routine. This allows the engine to load dynamic map blocks in the background while keeping the main frame loop and interrupt-driven audio playback threads running smoothly.
- Progression tracking is consolidated into a compact event flag matrix, minimizing memory

usage. These flags are evaluated in real-time during map transitions, allowing the engine to selectively stream only the required 3D models and character assets into the active heap.

- CPU cycle stalls are minimized by using the computationally simple LZSS0 compression algorithm, allowing real-time asset decompression to run alongside the game's core gameplay processes.

By combining these memory-saving techniques, Heart Beat engineered a highly efficient platform for its scale. This dynamic allocation system allowed the studio to bypass the hardware limitations of the fifth generation of consoles, fitting an exceptionally large and complex RPG world onto the PlayStation.

Works cited

1. I'm Loving Dragon Quest VII Reimagined, But There Is One Issue That Keeps Me From Enjoying As Much As I Would Like To... : r/dragonquest - Reddit, https://www.reddit.com/r/dragonquest/comments/1r3t4fm/im_loving_dragon_quest_vii_reimagined_but_there/
2. Dragon Quest VII: Sound Engine Analysis - GitHub Gist, <https://gist.github.com/loveemu/582ff7683a797b7048b6>
3. Dragon „Hackst“ IV | Markus Projects, <http://markus-projects.net/dragon-hackst-iv/>
4. How to Reverse-Engineer a PS1 Game – X-MAS CTF 2020 Writeup - GitHub, <https://gist.github.com/obskyr/99ce080f325bcc3d044f98fd90d447cb>
5. Lista de Juegos de Consola VCD | PDF - Scribd, <https://es.scribd.com/document/958885539/Lista-de-compatibilidad-oficial-de-POPStarter>
6. Exploring Tokimeki Memorial: Reverse-engineering PSX games with Ghidra, <https://tetracorp.github.io/tokimeki-memorial/methods/decompiling-psx-games.html>
7. In-Battle Cheat Codes for Characters | PDF | Video Games - Scribd, <https://www.scribd.com/document/896496426/SLUS-012-51>
8. PlayStation (PS1) Dragon Quest VII and IV Music Format (SEQQ) - loveemu labo, https://loveemu.hatenablog.com/entry/20140103/PlayStation_Dragon_Quest_Format
9. Dragon Warrior VII looks blurry when talking to NPCs or opening Menus - Reddit, https://www.reddit.com/r/dragonquest/comments/1ofokgu/dragon_warrior_vii_looks_blurry_when_talking_to/
10. DRAGON QUEST VII Reimagined Trainer Runtime Control Tool - GitHub, <https://github.com/DRAGON-QUEST-VII-Reimagined-Trainer>
11. Dragon Quest VII: Fragments of the Forgotten Past, https://dragon-quest.org/wiki/Dragon_Quest_VII:_Fragments_of_the_Forgotten_Past
12. Dragon Warrior/RAM map - Data Crystal - TCRF.net, https://datacrystal.tcrf.net/wiki/Dragon_Warrior/RAM_map
13. Kiefer - Dragon Quest Wiki - Fandom, <https://dragonquest.fandom.com/wiki/Kiefer>
14. Dragon Warrior VII - Immigrant Town FAQ - PlayStation - By Evanidus - GameFAQs, <https://gamefaqs.gamespot.com/ps/197152-dragon-warrior-vii/faqs/15425>
15. Man, game compression is a fuckin joke right now : r/TwoBestFriendsPlay - Reddit, https://www.reddit.com/r/TwoBestFriendsPlay/comments/1qa6l6b/man_game_compression_is_a_fuckin_joke_right_now/
16. Honestly, at this rate I'll have to uninstall it at Season 7 if it keeps hogging so much of my PC's memory, is this not too much? I know they COULD optimize the game if they wanted : r/marvelrivals - Reddit, https://www.reddit.com/r/marvelrivals/comments/1ptzru5/honestly_at_this_rate_ill_have_to_uninstall_it_at/
17. Dungeon Tileset - OpenGameArt.org |, <https://opengameart.org/content/dungeon-tileset-1>
18. Dungeon tileset - OpenGameArt.org |,

<https://opengameart.org/content/dungeon-tileset> 19. Category:Dragon Quest VII dungeons,
https://dragonquest.fandom.com/wiki/Category:Dragon_Quest_VII_dungeons 20. Why do some
people not like the DQ 7 3DS remake? : r/dragonquest - Reddit,
[https://www.reddit.com/r/dragonquest/comments/1at7dps/why_do_some_people_not_like_the_
dq_7_3ds_remake/](https://www.reddit.com/r/dragonquest/comments/1at7dps/why_do_some_people_not_like_the_dq_7_3ds_remake/)